C VARIABLES

The Complete Technical Manual

Volume 1.0 | Memory Management & Data Storage

01. Introduction to Variables

A variable in C is a named memory location used to store data that can change during program execution. Think of it as a labeled container in the computer's RAM.

> **Definition:** A variable is an identifier for a memory location that holds a value modifiable during runtime.

02. Why Variables Are Needed?

- **Dynamic Logic:** Allows programs to perform calculations.
- **User Interactivity:** Stores input provided by the user via `scanf()`.
- **Persistence:** Holds intermediate results during complex processing.

03. Variables and Memory

When you declare a variable, the computer reserves a specific number of bytes in RAM. Each variable possesses four key attributes:

1. **Name:** The identifier used in code.
2. **Data Type:** Determines memory size and operations allowed.
3. **Memory Address:** Where it physically resides.
4. **Value:** The actual data stored inside.

```
int age = 20; // 4 bytes reserved for 'age'
```

04. Declaration & Initialization

Declaration

Tells the compiler the variable name and type. No value is assigned yet.

```
data_type variable_name; // declaration
```

Initialization

Assigning an initial value to the declared variable.

```
int x = 10; // declaration + initialization
```

Phase	Effect
Declaration	Reserves Memory
Initialization	Fills Memory with Data

05. Types: Local & Global

Local Variables

Declared inside a function. They are "private" to that function and are destroyed once execution reaches the end of the block }.

Global Variables

Declared outside all functions. They are "public" and accessible by any part of the program from the point of declaration until the program ends.

06. Static Variables

Static variables are unique. While their scope is local, their **lifetime** is global. They retain their value even after the function finishes.

```
void counter() {
  static int count = 0;
  count++;
  printf("%d", count);
}
```

Calling `counter()` three times will print "1 2 3" because `count` is not reset to 0.

07. Storage Classes

Automatic (auto): The default for all local variables. They live on the "Stack".

External (extern): Used to reference a variable that is defined in another file. This is crucial for multi-file projects.

Keywords: `auto`, `extern`

08. Rules for Identifiers

To write valid C code, your variable names must follow these strict rules:

- Must start with a letter (a-z) or underscore (_).
- Symbols and spaces are strictly forbidden.
- Cannot start with a numeric digit.
- Cannot be a Keyword (like `int` or `return`).

✅ Correct: `_sum`, `totalAmount`
❌ Wrong: `1st_place`, `int`

09. Case Sensitivity & Constants

In C, `int num;` and `int NUM;` are two independent variables.

Constants (const)

To prevent a variable's value from being changed, use the `const` keyword. This turns the variable into a read-only entity.

```
const float PI = 3.14159;
```

10. Sizes and Limits

Type	Typical Size	Common Specifier
int	4 Bytes	%d
float	4 Bytes	%f
double	8 Bytes	%lf
char	1 Byte	%c

Note: Sizes may vary depending on the compiler and architecture (32-bit vs 64-bit).

11. Scope vs Lifetime

Scope: The region of the program where a variable is visible/accessible (Block, Function, or File).

Lifetime: The duration for which the variable remains in memory during execution.

Local variables have Block Scope, while Global variables have File Scope.

12. Advanced Concepts

Register Variables

Requesting the compiler to store a variable in the CPU register instead of RAM for lightning-fast access (used in loops).

Variable Shadowing

When a local variable has the same name as a global variable, the local one "shadows" or hides the global one within that function.

13. Input & Output with Variables

To interact with variables, we use `printf` for output and `scanf` for input.

```
int age;
printf("Enter age: ");
scanf("%d", &age); // & is the Address Operator
printf("Your age is %d", age);
```

14. Troubleshooting Common Errors

- **Usage before declaration:** C is top-down; declare first!
- **Garbage Values:** Accessing uninitialized local variables.
- **Format Mismatch:** Using %d for a `float` variable.
- **The Semicolon:** Forgetting ; at the end of declaration.

15. Summary & Best Practices

✅ Use camelCase or snake_case for readability.
✅ Always initialize variables to avoid garbage values.
✅ Limit global variables to prevent data corruption.

Key Interview Question: What is a static variable?
Answer: A variable that retains its value between function calls and lives for the entire program duration.

Programmers don't die, they just lose their scope.